

AMITY UNIVERSITY NOIDA

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY

JAVA PROGRAMMING

PRACTICAL FILE

Made By-

Shourya Solanki

A2305223569

B.Tech 4CSE-3Y

Submitted To-

Dr Subhash Chand Gupta

1) WAP to find greatest of three numbers.

Code:

```
package com.mycompany.greatest;

import java.util.Scanner;

public class Greatest
{
   public static void main(String[] args)
   {
      Scanner scanner = new Scanner(System.in);

      System.out.println("Enter first number: ");
      int num1 = scanner.nextInt();

      System.out.println("Enter second number: ");
      int num2 = scanner.nextInt();

      System.out.println("Enter third number: ");
      int num3 = scanner.nextInt();

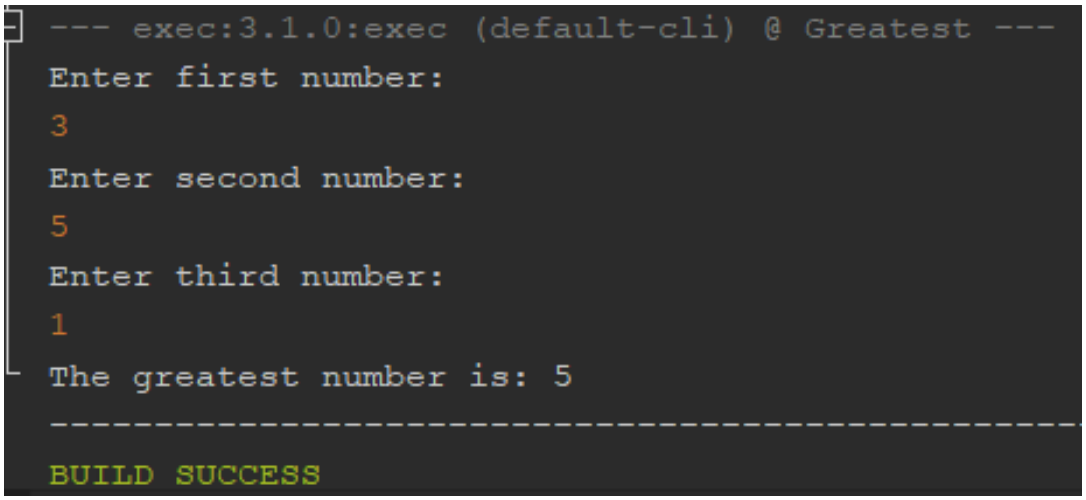
      int greatest;
      if(num1 >= num2 && num1 >= num3)
      {
         greatest = num1;
      }
      else if (num2 >= num1 && num2 >= num3)
      {
```

```
            greatest = num2;
        }
        else
        {
            greatest = num3;
        }

        System.out.println("The greatest number is: " + greatest);

        scanner.close();
    }
}
```

OUTPUT



2) WAP to calculate factorial of a number using command line arguments.

Code:

```
package com.mycompany.factorial;


public class Factorial
{


   public static void main(String[] args)
   {
      if (args.length == 0)
      {
         System.out.println("Please provide a number as a command line
argument.");
         return;
      }

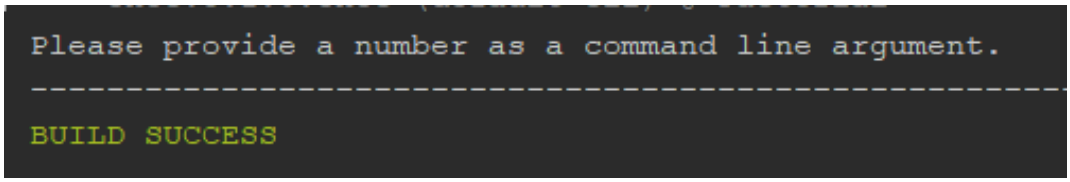

      try
      {
         int num = Integer.parseInt(args[0]);


         long factorial = 1;
         for (int i = 1; i <= num; i++)
         {
            factorial *= i;
         }


         System.out.println("Factorial of " + num + " is: " + factorial);
      }
      catch (NumberFormatException e)
```

```
        {
            System.out.println("Invalid input! Please enter a valid integer.");
        }
    }
}
```

OUTPUT



3) WAP to read a set of numbers in an array & to find the sum and average them.

Code:

```
package com.mycompany.sumavg;


import java.util.Scanner;


public class SumAvg
{
   public static void main(String[] args)
   {
      Scanner scanner = new Scanner(System.in);


      System.out.print("Enter the number of elements: ");
      int n = scanner.nextInt();

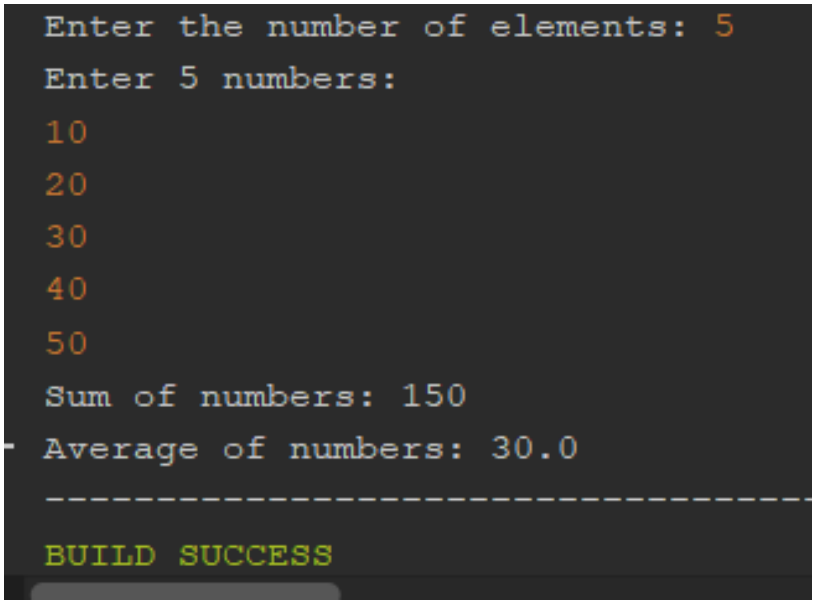

      int[] numbers = new int[n];


      System.out.println("Enter " + n + " numbers:");
      int sum = 0;
      for (int i = 0; i < n; i++)
      {
         numbers[i] = scanner.nextInt();
         sum += numbers[i];
      }
      double average = (double) sum / n;
```

```
        System.out.println("Sum of numbers: " + sum);
        System.out.println("Average of numbers: " + average);

        scanner.close();
    }
}
```

OUTPUT



4) WAP to maintain the student record containing roll number, Name, marks1, marks2, marks3 as data member and getdata(), display() and setdata() as member functions.

Code:

```
import java.util.Scanner;


class Student {
    private int rollNumber;
    private String name;
    private int marks1, marks2, marks3;


    public void getData() {
        Scanner scanner = new Scanner(System.in);


        System.out.print("Enter Roll Number: ");
        rollNumber = scanner.nextInt();
        scanner.nextLine(); // Consume newline


        System.out.print("Enter Name: ");
        name = scanner.nextLine();


        System.out.print("Enter Marks in Subject 1: ");
        marks1 = scanner.nextInt();


        System.out.print("Enter Marks in Subject 2: ");
        marks2 = scanner.nextInt();


        System.out.print("Enter Marks in Subject 3: ");
```

```
        marks3 = scanner.nextInt();
    }

    public void display() {
        System.out.println("\nStudent Record:");
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Marks 1: " + marks1);
        System.out.println("Marks 2: " + marks2);
        System.out.println("Marks 3: " + marks3);
    }

    public void setData(int roll, String studentName, int m1, int m2, int m3) {
        rollNumber = roll;
        name = studentName;
        marks1 = m1;
        marks2 = m2;
        marks3 = m3;
        System.out.println("\nStudent details updated successfully!");
    }
}

public class StudentRecord {
    public static void main(String[] args) {
        // Creating a student object
        Student student = new Student();
```

```
        // Getting data from user
        student.getData();

        // Displaying student details
        student.display();

        // Updating student details (Example)
        student.setData(102, "John Doe", 85, 90, 88);

        // Displaying updated details
        student.display();
    }
}
```

OUTPUT

```
Enter Roll Number: 101
Enter Name: Alice
Enter Marks in Subject 1: 78
Enter Marks in Subject 2: 82
Enter Marks in Subject 3: 89

Student Record:
Roll Number: 101
Name: Alice
Marks 1: 78
Marks 2: 82
Marks 3: 89

Student details updated successfully!

Student Record:
Roll Number: 102
Name: John Doe
Marks 1: 85
Marks 2: 90
Marks 3: 88
```

5) WAP to increment the employee salaries on the basis of their designation (manager-5000, general manager-10000, ceo-20000, worker-2000). Use employee named, id, designation, salary as data member and inc_sal as member function.

Code:

```
import java.util.Scanner;


class Employee {
   // Data members
   private int empId;
   private String name, designation;
   private double salary;


   public Employee(int empId, String name, String designation, double salary) {
      this.empId = empId;
      this.name = name;
      this.designation = designation;
      this.salary = salary;
   }
   public void incSal() {
      int increment = 0;


      switch (designation.toLowerCase()) {
         case "manager":
            increment = 5000;
            break;
         case "general manager":
```

```
            increment = 10000;
            break;
        case "ceo":
            increment = 20000;
            break;
        case "worker":
            increment = 2000;
            break;
        default:
            System.out.println("Invalid designation. No increment applied.");
            return;
    }

    salary += increment;
    System.out.println("\nSalary incremented by " + increment);
  }
  public void display() {
    System.out.println("\nEmployee Details:");
    System.out.println("Employee ID: " + empId);
    System.out.println("Name: " + name);
    System.out.println("Designation: " + designation);
    System.out.println("Updated Salary: " + salary);
  }
}
public class EmployeeSalary {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
```

```
        System.out.print("Enter Employee ID: ");
        int empId = scanner.nextInt();
        scanner.nextLine(); // Consume newline

        System.out.print("Enter Name: ");
        String name = scanner.nextLine();

        System.out.print("Enter Designation (manager/general
manager/ceo/worker): ");
        String designation = scanner.nextLine();

        System.out.print("Enter Current Salary: ");
        double salary = scanner.nextDouble();
        Employee emp = new Employee(empId, name, designation, salary);
        emp.incSal();

        emp.display();

        scanner.close();
    }
}
```

OUTPUT

```
Enter Employee ID: 101
Enter Name: John
Enter Designation (manager/general manager/ceo/worker): manager
Enter Current Salary: 50000

Salary incremented by 5000

Employee Details:
Employee ID: 101
Name: John
Designation: manager
Updated Salary: 55000.0
```

6) Write a class bank, containing data member: Name of Depositor, A/c type, Type of A/c, Balance amount. Member function: To assign initial value, To deposit an amount , to withdraw an amount after checking the balance (which should be greater than Rs. 500) , To display name & balance.

Code:

```
import java.util.Scanner;


class Bank {
    private String depositorName;
    private String accType;
    private int accNumber;
    private double balance;


    public void assignInitialValue(String name, int accNo, String type, double initialBalance) {
        depositorName = name;
        accNumber = accNo;
        accType = type;
        balance = initialBalance;
    }


    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("₹" + amount + " deposited successfully!");
        } else {
            System.out.println("Invalid deposit amount!");
        }
```

```
    }

    public void withdraw(double amount) {
        if (amount > 0 && (balance - amount) >= 500) {
            balance -= amount;
            System.out.println("₹" + amount + " withdrawn successfully!");
        } else {
            System.out.println("Insufficient balance! Minimum balance of ₹500 must be maintained.");
        }
    }

    public void display() {
        System.out.println("\nAccount Details:");
        System.out.println("Depositor Name: " + depositorName);
        System.out.println("Account Number: " + accNumber);
        System.out.println("Account Type: " + accType);
        System.out.println("Current Balance: ₹" + balance);
    }
}

public class BankAccount {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        Bank account = new Bank();
```

```
        System.out.print("Enter Depositor Name: ");
        String name = scanner.nextLine();

        System.out.print("Enter Account Number: ");
        int accNo = scanner.nextInt();
        scanner.nextLine(); // Consume newline

        System.out.print("Enter Account Type (Savings/Current): ");
        String accType = scanner.nextLine();

        System.out.print("Enter Initial Balance: ₹");
        double initialBalance = scanner.nextDouble();
        account.assignInitialValue(name, accNo, accType, initialBalance);
        account.display();
        System.out.print("\nEnter amount to deposit: ₹");
        double depositAmount = scanner.nextDouble();
        account.deposit(depositAmount);
        account.display();
        System.out.print("\nEnter amount to withdraw: ₹");
        double withdrawAmount = scanner.nextDouble();
        account.withdraw(withdrawAmount);
        account.display();
        scanner.close();
    }
}
```

OUTPUT

```
Enter Depositor Name: Rahul Sharma
Enter Account Number: 123456
Enter Account Type (Savings/Current): Savings
Enter Initial Balance: ₹1000

Account Details:
Depositor Name: Rahul Sharma
Account Number: 123456
Account Type: Savings
Current Balance: ₹1000.0

Enter amount to deposit: ₹2000
₹2000.0 deposited successfully!

Account Details:
Depositor Name: Rahul Sharma
Account Number: 123456
Account Type: Savings
Current Balance: ₹3000.0
```

```
Enter amount to withdraw: ₹2500
Insufficient balance! Minimum balance of ₹500 must be maintained.

Account Details:
Depositor Name: Rahul Sharma
Account Number: 123456
Account Type: Savings
Current Balance: ₹3000.0
```

7) Create a class Employee. Derive 3 classes from this class namely, Programmer, Analyst & Project Leader. Take attributes and operations on your own.

Code:

```
import java.util.Scanner;


class Employee {
    protected int empId;
    protected String name;
    protected double salary;

    public Employee(int empId, String name, double salary) {
        this.empId = empId;
        this.name = name;
        this.salary = salary;
    }

    public void displayDetails() {
        System.out.println("Employee ID: " + empId);
        System.out.println("Name: " + name);
        System.out.println("Salary: ₹" + salary);
    }
}

class Programmer extends Employee {
    private String programmingLanguage;
```

```
    public Programmer(int empId, String name, double salary, String
programmingLanguage) {
        super(empId, name, salary);
        this.programmingLanguage = programmingLanguage;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Role: Programmer");
        System.out.println("Programming Language: " +
programmingLanguage);
    }
}

class Analyst extends Employee {
    private String domain;

    public Analyst(int empId, String name, double salary, String domain) {
        super(empId, name, salary);
        this.domain = domain;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Role: Analyst");
        System.out.println("Specialized Domain: " + domain);
```

```
    }
}

class ProjectLeader extends Employee {
    private int teamSize;

    public ProjectLeader(int empId, String name, double salary, int teamSize)
{
        super(empId, name, salary);
        this.teamSize = teamSize;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Role: Project Leader");
        System.out.println("Team Size: " + teamSize);
    }
}

public class EmployeeHierarchy {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Programmer ID: ");
        int pId = scanner.nextInt();
        scanner.nextLine();
```

```
        System.out.print("Enter Programmer Name: ");
        String pName = scanner.nextLine();
        System.out.print("Enter Programmer Salary: ₹");
        double pSalary = scanner.nextDouble();
        scanner.nextLine();
        System.out.print("Enter Programming Language: ");
        String language = scanner.nextLine();

        Programmer programmer = new Programmer(pId, pName, pSalary,
language);

        System.out.print("\nEnter Analyst ID: ");
        int aId = scanner.nextInt();
        scanner.nextLine();
        System.out.print("Enter Analyst Name: ");
        String aName = scanner.nextLine();
        System.out.print("Enter Analyst Salary: ₹");
        double aSalary = scanner.nextDouble();
        scanner.nextLine();
        System.out.print("Enter Specialized Domain: ");
        String domain = scanner.nextLine();

        Analyst analyst = new Analyst(aId, aName, aSalary, domain);

        System.out.print("\nEnter Project Leader ID: ");
        int plId = scanner.nextInt();
        scanner.nextLine();
```

```
        System.out.print("Enter Project Leader Name: ");
        String plName = scanner.nextLine();
        System.out.print("Enter Project Leader Salary: ₹");
        double plSalary = scanner.nextDouble();
        System.out.print("Enter Team Size: ");
        int teamSize = scanner.nextInt();

        ProjectLeader projectLeader = new ProjectLeader(plId, plName,
plSalary, teamSize);

        System.out.println("\n--------------------------------");
        programmer.displayDetails();
        System.out.println("--------------------------------");
        analyst.displayDetails();
        System.out.println("--------------------------------");
        projectLeader.displayDetails();
        System.out.println("--------------------------------");

        scanner.close();
    }
}
```

OUTPUT

```
Enter Programmer ID: 101
Enter Programmer Name: John Doe
Enter Programmer Salary: ₹50000
Enter Programming Language: Java

Enter Analyst ID: 102
Enter Analyst Name: Alice Smith
Enter Analyst Salary: ₹60000
Enter Specialized Domain: Data Analysis

Enter Project Leader ID: 103
Enter Project Leader Name: Robert Brown
Enter Project Leader Salary: ₹80000
Enter Team Size: 10
```


```
Enter Programmer ID: 101
Enter Programmer Name: John Doe
Enter Programmer Salary: ₹50000
Enter Programming Language: Java

Enter Analyst ID: 102
Enter Analyst Name: Alice Smith
Enter Analyst Salary: ₹60000
Enter Specialized Domain: Data Analysis

Enter Project Leader ID: 103
Enter Project Leader Name: Robert Brown
Enter Project Leader Salary: ₹80000
Enter Team Size: 10
```


8) Design three classes: Student, Exam and Result. The student class has data members such as roll no, name etc. Create a class Exam by inheriting

the Student class. The Exam class adds data members representing the marks scored in six subjects. Derive the Result from class Exam and it has its own members such as total marks and average. Calculate the total marks and average.

Code:

```
import java.util.Scanner;


class Student {
    protected int rollNo;
    protected String name;

    public Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    public void displayDetails() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }
}

class Exam extends Student {
    protected int[] marks = new int[6];

    public Exam(int rollNo, String name, int[] marks) {
        super(rollNo, name);
        this.marks = marks;
```

```
    }

    public void displayMarks() {
        System.out.println("Marks in 6 subjects:");
        for (int i = 0; i < 6; i++) {
            System.out.println("Subject " + (i + 1) + ": " + marks[i]);
        }
    }
}

class Result extends Exam {
    private int totalMarks;
    private double average;

    public Result(int rollNo, String name, int[] marks) {
        super(rollNo, name, marks);
        calculateResult();
    }

    private void calculateResult() {
        totalMarks = 0;
        for (int mark : marks) {
            totalMarks += mark;
        }
        average = totalMarks / 6.0;
    }
```

```
    public void displayResult() {
        displayDetails();
        displayMarks();
        System.out.println("Total Marks: " + totalMarks);
        System.out.println("Average Marks: " + average);
    }
}

public class StudentExamResult {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Roll No: ");
        int rollNo = scanner.nextInt();
        scanner.nextLine();
        System.out.print("Enter Name: ");
        String name = scanner.nextLine();

        int[] marks = new int[6];
        System.out.println("Enter marks for 6 subjects:");
        for (int i = 0; i < 6; i++) {
            System.out.print("Subject " + (i + 1) + ": ");
            marks[i] = scanner.nextInt();
        }

        Result studentResult = new Result(rollNo, name, marks);
        System.out.println("\n---------- Student Result ----------");
```

```
        studentResult.displayResult();

        scanner.close();
    }
}
```

OUTPUT

```
Enter Roll No: 101
Enter Name: Rahul Sharma
Enter marks for 6 subjects:
Subject 1: 85
Subject 2: 90
Subject 3: 78
Subject 4: 88
Subject 5: 76
Subject 6: 95

---------- Student Result ----------
Roll No: 101
Name: Rahul Sharma
Marks in 6 subjects:
Subject 1: 85
Subject 2: 90
Subject 3: 78
Subject 4: 88
Subject 5: 76
Subject 6: 95
Total Marks: 512
Average Marks: 85.33333333333333
```

9) Calculate area of different geometrical figures(circle, rectangle,square, triangle) using function overloading.

Code:

```
import java.util.Scanner;

class Geometry {

    public double area(double radius) {
        return Math.PI * radius * radius;
    }

    public double area(double length, double width) {
        return length * width;
    }

    public double area(int side) {
        return side * side;
    }

    public double area(double base, double height, boolean isTriangle) {
        if (isTriangle) {
            return 0.5 * base * height;
        }
        return 0;
    }
}

public class GeometricAreaCalculator {
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
        Geometry geometry = new Geometry();

        System.out.print("Enter radius of the circle: ");
        double radius = scanner.nextDouble();
        System.out.println("Area of Circle: " + geometry.area(radius));

        System.out.print("\nEnter length and width of the rectangle: ");
        double length = scanner.nextDouble();
        double width = scanner.nextDouble();
        System.out.println("Area of Rectangle: " + geometry.area(length,
width));

        System.out.print("\nEnter side of the square: ");
        int side = scanner.nextInt();
        System.out.println("Area of Square: " + geometry.area(side));

        System.out.print("\nEnter base and height of the triangle: ");
        double base = scanner.nextDouble();
        double height = scanner.nextDouble();
        System.out.println("Area of Triangle: " + geometry.area(base, height,
true));
        scanner.close();
    }
}
```

OUTPUT

```
Enter radius of the circle: 5
Area of Circle: 78.53981633974483

Enter length and width of the rectangle: 10 20
Area of Rectangle: 200.0

Enter side of the square: 6
Area of Square: 36.0

Enter base and height of the triangle: 8 12
Area of Triangle: 48.0
```

10) WAP to implement multiple inheritance using Interface.

Code:

```
import java.util.Scanner;

interface Person {
   void displayPersonalInfo();
}

interface Employee {
   void displayEmployeeInfo();
}

class Manager implements Person, Employee {
   private String name;
   private int age;
   private int empId;
   private double salary;

   public Manager(String name, int age, int empId, double salary) {
      this.name = name;
      this.age = age;
      this.empId = empId;
      this.salary = salary;
   }

   public void displayPersonalInfo() {
      System.out.println("Name: " + name);
```

```
        System.out.println("Age: " + age);
    }

    public void displayEmployeeInfo() {
        System.out.println("Employee ID: " + empId);
        System.out.println("Salary: ₹" + salary);
    }

    public void displayDetails() {
        displayPersonalInfo();
        displayEmployeeInfo();
    }
}

public class MultipleInheritanceDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Name: ");
        String name = scanner.nextLine();

        System.out.print("Enter Age: ");
        int age = scanner.nextInt();

        System.out.print("Enter Employee ID: ");
        int empId = scanner.nextInt();
```

```
        System.out.print("Enter Salary: ₹");
        double salary = scanner.nextDouble();

        Manager manager = new Manager(name, age, empId, salary);

        System.out.println("\n----- Manager Details -----");
        manager.displayDetails();

        scanner.close();
    }
}
```

OUTPUT

```
Enter Name: Rajesh Sharma
Enter Age: 40
Enter Employee ID: 101
Enter Salary: ₹75000

----- Manager Details -----
Name: Rajesh Sharma
Age: 40
Employee ID: 101
Salary: ₹75000.0
```

11) WAP to handle ArithmaticEception and ArrayIndexOutOfBounds Exception.

Code:

```
import java.util.Scanner;


public class ExceptionHandlingDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);


        // Handling ArithmeticException
        try {
            System.out.print("Enter numerator: ");
            int numerator = scanner.nextInt();
            System.out.print("Enter denominator: ");
            int denominator = scanner.nextInt();
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");
        }


        // Handling ArrayIndexOutOfBoundsException
        try {
            int[] numbers = {10, 20, 30, 40, 50};
            System.out.print("\nEnter index to access: ");
            int index = scanner.nextInt();
            System.out.println("Value at index " + index + ": " +
numbers[index]);
```

```
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Index out of bounds. Please enter a valid index.");
        }

        System.out.println("\nProgram execution completed.");
        scanner.close();
    }
}
```

OUTPUT

```
Enter numerator: 10
Enter denominator: 0
Error: Division by zero is not allowed.

Enter index to access: 2
Value at index 2: 30

Program execution completed.
```

```
Enter numerator: 20
Enter denominator: 5
Result: 4

Enter index to access: 10
Error: Index out of bounds. Please enter a valid index.

Program execution completed.
```

12) WAP to create and handle your own Exception.

Code:

```
import java.util.Scanner;

// Custom Exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

class VotingEligibility {
    public static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above to vote.");
        } else {
            System.out.println("You are eligible to vote.");
        }
    }
}

public class CustomExceptionDemo {
    public static
```

OUTPUT

```
Enter your age: 16
Exception Caught: Age must be 18 or above to vote.

Program execution completed.
```

```
Enter your age: 20
You are eligible to vote.

Program execution completed.
```

13) WAP to create Student class in package1 and Marks class in package2 which inherit Student class. Calculate the total and average of marks in Result class.

Code:

```
package package1;


public class Student {
    protected int rollNo;
    protected String name;

    public Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    public void displayStudentDetails() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }
}

package package2;
import package1.Student;

public class Marks extends Student {
    protected int[] marks = new int[5];

    public Marks(int rollNo, String name, int[] marks) {
```

```
        super(rollNo, name);
        this.marks = marks;
    }

    public void displayMarks() {
        System.out.println("Marks in 5 subjects:");
        for (int i = 0; i < 5; i++) {
            System.out.println("Subject " + (i + 1) + ": " + marks[i]);
        }
    }
}

package package2;

public class Result extends Marks {
    private int totalMarks;
    private double average;

    public Result(int rollNo, String name, int[] marks) {
        super(rollNo, name, marks);
        calculateResult();
    }

    private void calculateResult() {
        totalMarks = 0;
        for (int mark : marks) {
            totalMarks += mark;
```

```
        }
        average = totalMarks / 5.0;
    }

    public void displayResult() {
        displayStudentDetails();
        displayMarks();
        System.out.println("Total Marks: " + totalMarks);
        System.out.println("Average Marks: " + average);
    }
}

import package2.Result;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Roll No: ");
        int rollNo = scanner.nextInt();
        scanner.nextLine();

        System.out.print("Enter Name: ");
        String name = scanner.nextLine();

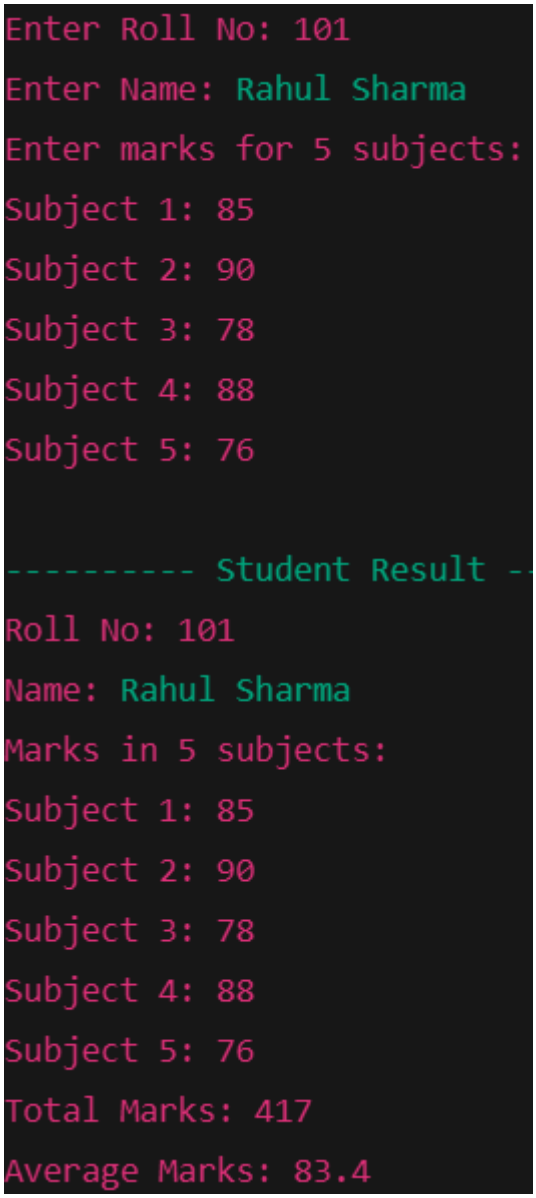
        int[] marks = new int[5];
```

```
        System.out.println("Enter marks for 5 subjects:");
        for (int i = 0; i < 5; i++) {
            System.out.print("Subject " + (i + 1) + ": ");
            marks[i] = scanner.nextInt();
        }

        Result studentResult = new Result(rollNo, name, marks);
        System.out.println("\n---------- Student Result ----------");
        studentResult.displayResult();

        scanner.close();
    }
}
```

OUTPUT



15) Write a program to show concept of multiple inheritance through implementation of interfaces in a class.

Code:

```
import java.util.Scanner;

interface Person {
   void displayPersonalInfo();
}

interface Employee {
   void displayEmployeeInfo();
}

class Manager implements Person, Employee {
   private String name;
   private int age;
   private int empId;
   private double salary;

   public Manager(String name, int age, int empId, double salary) {
      this.name = name;
      this.age = age;
      this.empId = empId;
      this.salary = salary;
   }

   public void displayPersonalInfo() {
      System.out.println("Name: " + name);
      System.out.println("Age: " + age);
```

```
    }

    public void displayEmployeeInfo() {
        System.out.println("Employee ID: " + empId);
        System.out.println("Salary: ₹" + salary);
    }

    public void displayDetails() {
        displayPersonalInfo();
        displayEmployeeInfo();
    }
}

public class MultipleInheritanceDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Name: ");
        String name = scanner.nextLine();

        System.out.print("Enter Age: ");
        int age = scanner.nextInt();

        System.out.print("Enter Employee ID: ");
        int empId = scanner.nextInt();

        System.out.print("Enter Salary: ₹");
```

```
        double salary = scanner.nextDouble();

        Manager manager = new Manager(name, age, empId, salary);

        System.out.println("\n----- Manager Details -----");
        manager.displayDetails();

        scanner.close();
    }
}
```

OUTPUT

```
Enter Name: Rajesh Sharma
Enter Age: 40
Enter Employee ID: 101
Enter Salary: ₹75000

----- Manager Details -----
Name: Rajesh Sharma
Age: 40
Employee ID: 101
Salary: ₹75000.0
```

16) Write a program with given interfaces MotorBike and Cycle, then implement in child class TwoWheeler and display distance & speed.

Code:

```
import java.util.Scanner;

interface MotorBike {
   int maxSpeed = 150; // km/h
   void displayBikeInfo();
}

interface Cycle {
   int maxSpeed = 40; // km/h
   void displayCycleInfo();
}

class TwoWheeler implements MotorBike, Cycle {
   private double distance;
   private double speed;

   public TwoWheeler(double distance, double speed) {
      this.distance = distance;
      this.speed = speed;
   }

   public void displayBikeInfo() {
      System.out.println("MotorBike - Max Speed: " + MotorBike.maxSpeed
+ " km/h");
   }
```

```
    public void displayCycleInfo() {
        System.out.println("Cycle - Max Speed: " + Cycle.maxSpeed + " km/h");
    }

    public void displayDetails() {
        displayBikeInfo();
        displayCycleInfo();
        System.out.println("Actual Distance Traveled: " + distance + " km");
        System.out.println("Current Speed: " + speed + " km/h");
    }
}

public class MultipleInheritanceExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Distance Traveled (in km): ");
        double distance = scanner.nextDouble();

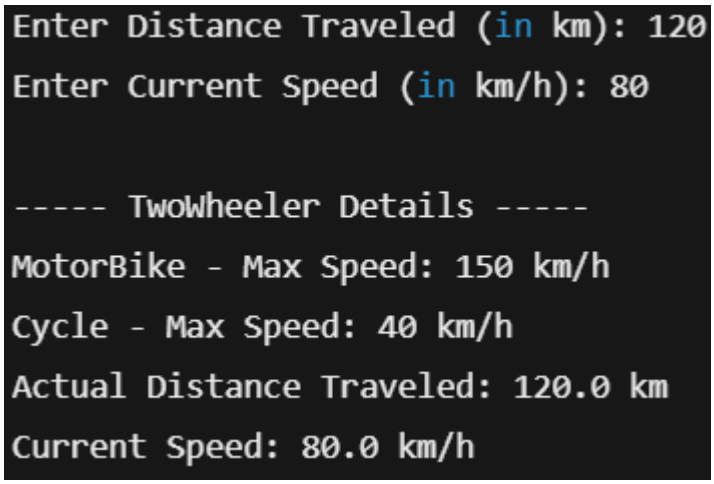
        System.out.print("Enter Current Speed (in km/h): ");
        double speed = scanner.nextDouble();

        TwoWheeler vehicle = new TwoWheeler(distance, speed);

        System.out.println("\n----- TwoWheeler Details -----");
        vehicle.displayDetails();
```

```
        scanner.close();
    }
}
```

OUTPUT



17) Write a program in Java to create an immutable class.

Code:

```
final class ImmutableStudent {
    private final int rollNo;
    private final String name;

    public ImmutableStudent(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    public int getRollNo() {
        return rollNo;
    }

    public String getName() {
        return name;
    }

    public String toString() {
        return "Student [Roll No: " + rollNo + ", Name: " + name + "]";
    }
}

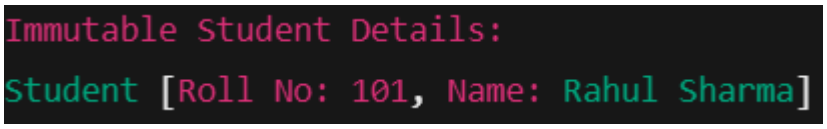
public class ImmutableClassExample {
    public static void main(String[] args) {
```

```
        ImmutableStudent student = new ImmutableStudent(101, "Rahul
Sharma");


        System.out.println("Immutable Student Details:");
        System.out.println(student);


        // Trying to modify (Not possible)
        // student.rollNo = 102;  // Compilation error
        // student.name = "Amit"; // Compilation error
    }
}
```

OUTPUT



18) Write a program in Java to use final variables, final methods and final classes.

Code:

```
// Final class cannot be inherited
final class FinalClass {
    // Final variable (constant)
    private final double PI = 3.14159;

    // Final method cannot be overridden
    public final void displayValue() {
        System.out.println("Final Variable PI: " + PI);
    }
}

// Subclass attempt (Uncommenting will cause error)
// class SubClass extends FinalClass { } // Error: Cannot inherit from final class

class ParentClass {
    public final void showMessage() {
        System.out.println("This is a final method from ParentClass.");
    }
}

class ChildClass extends ParentClass {
    // Trying to override final method (Uncommenting will cause error)
    // public void showMessage() {
    //     System.out.println("Overriding final method.");
```

```
    // }
}

public class FinalKeywordExample {
    public static void main(String[] args) {
        FinalClass obj = new FinalClass();
        obj.displayValue();

        ChildClass child = new ChildClass();
        child.showMessage();

        // Final variable example
        final int number = 100;
        System.out.println("Final Variable Number: " + number);

        // Attempting to modify final variable (Uncommenting will cause error)
        // number = 200; // Error: cannot assign a value to final variable 'number'
    }
}
```

OUTPUT

```
Final Variable PI: 3.14159
This is a final method from ParentClass.
Final Variable Number: 100
```

19) Write a program in Java to create thread by extending Thread class.

Code:

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + " is
running: " + i);
            try {
                Thread.sleep(1000); // Pause for 1 second
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted: " + e.getMessage());
            }
        }
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread thread1 = new MyThread();
        MyThread thread2 = new MyThread();

        thread1.setName("Thread 1");
        thread2.setName("Thread 2");
        thread1.start(); // Start the first thread
        thread2.start(); // Start the second thread
    }
}
```

OUTPUT

```
Thread 1 is running: 1
Thread 2 is running: 1
Thread 1 is running: 2
Thread 2 is running: 2
Thread 1 is running: 3
Thread 2 is running: 3
Thread 1 is running: 4
Thread 2 is running: 4
Thread 1 is running: 5
Thread 2 is running: 5
```

20) Write a program in Java to create thread by implementing Runnable interface.

Code:

```
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + " is running: " + i);
            try {
                Thread.sleep(1000); // Pause for 1 second
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted: " + e.getMessage());
            }
        }
    }
}

public class RunnableExample {
    public static void main(String[] args) {
        MyRunnable myRunnable = new MyRunnable();
        Thread thread1 = new Thread(myRunnable, "Thread 1");
        Thread thread2 = new Thread(myRunnable, "Thread 2");

        thread1.start(); // Start the first thread
        thread2.start(); // Start the second thread
    }
}
```

OUTPUT

```
Thread 1 is running: 1
Thread 2 is running: 1
Thread 1 is running: 2
Thread 2 is running: 2
Thread 1 is running: 3
Thread 2 is running: 3
Thread 1 is running: 4
Thread 2 is running: 4
Thread 1 is running: 5
Thread 2 is running: 5
```

21) Write a program to create a file in Java using FileOutputStream class and enter byte data into it.

Code:

```
import java.io.FileOutputStream;

import java.io.IOException;


public class FileOutputStreamExample {

   public static void main(String[] args) {

      String data = "Hello, this is a test file created using FileOutputStream in Java.";


      try {

         FileOutputStream fos = new FileOutputStream("output.txt"); // Creating a file

         byte[] byteData = data.getBytes(); // Converting string to byte array

         fos.write(byteData); // Writing byte data to file

         fos.close(); // Closing the stream

         System.out.println("File created successfully and data written.");

      } catch (IOException e) {

         System.out.println("An error occurred: " + e.getMessage());

      }

   }

}
```

OUTPUT

```
File created successfully and data written.
```

```
Hello, this is a test file created using FileOutputStream in Java.
```

22) Write a program to create a file in Java using FileOutputStream class and enter String data into it.

Code:

```
import java.io.FileOutputStream;

import java.io.IOException;


public class FileWriteExample {

   public static void main(String[] args) {

      String data = "Hello, this is a sample text written using FileOutputStream in Java.";


      try {

         FileOutputStream fos = new FileOutputStream("sample.txt"); // Creating a file

         fos.write(data.getBytes()); // Converting string to byte array and writing to file

         fos.close(); // Closing the file stream

         System.out.println("File created successfully and data written.");

      } catch (IOException e) {

         System.out.println("An error occurred: " + e.getMessage());

      }

   }

}
```

OUTPUT

```
File created successfully and data written.
```

```
Hello, this is a sample text written using FileOutputStream in Java.
```

23) Open a file using FileInputStream, read its content and siplay on screen. (Remember to create a file in Java using FileOutputStream class and enter data into it before.

Code:

```
import java.io.FileOutputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class FileReadWriteExample {
    public static void main(String[] args) {
        String fileName = "example.txt";
        String data = "Hello, this is a test file. Learning File Handling in Java!";

        // Step 1: Writing data to the file
        try {
            FileOutputStream fos = new FileOutputStream(fileName);
            fos.write(data.getBytes());
            fos.close();
            System.out.println("Data written to file successfully.");
        } catch (IOException e) {
            System.out.println("Error writing file: " + e.getMessage());
            return;
        }

        // Step 2: Reading data from the file
        try {
            FileInputStream fis = new FileInputStream(fileName);
            int content;
```

```
            System.out.println("Reading file content:");
            while ((content = fis.read()) != -1) {
                System.out.print((char) content); // Converting byte to character and displaying
            }
            fis.close();
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
Data written to file successfully.
Reading file content:
Hello, this is a test file. Learning File Handling in Java!
```

```
Hello, this is a test file. Learning File Handling in Java!
```